

Aluna: Bianca de Moraes Gomes

Matrícula: 376131

Curso: I.A para devs

Link para o repositório no Github: https://github.com/bmgbianca/tech_challenge_fase_1

Link para o vídeo do Youtube: <https://youtu.be/bULDELxLMPA>

TECH CHALLENGE – FASE 1

1. INTRODUÇÃO

1.1. Objetivo:

O presente estudo tem por objetivo treinar e avaliar modelos classificadores de **machine learning** capazes de classificar uma determinada paciente como portadora da **Síndrome dos Ovários Policísticos (SOP)** ou não, a partir da análise de dados clínicos, laboratoriais e de estilo de vida da mesma. Este estudo faz parte do projeto de conclusão da fase 1 (aqui denominado *tech challenge*) da pós-graduação em I.A para devs, oferecida pela Faculdade de Informática e Administração Paulista (FIAP).

1.2. Escolha da base de dados:

Para o estudo, foi escolhida uma base de dados (neste documento também denominada como *dataset*) que pudesse oferecer a possibilidade de aplicação da maioria dos métodos e conceitos aprendidos durante as aulas, como codificação de variáveis categóricas, escalonamento de variáveis numéricas e aplicação de método para redução de dimensionalidade. Seguindo a determinação de selecionar uma base de dados relacionada à saúde e segurança da mulher, esta base foi escolhida por ser uma das únicas encontradas a não ter seus dados já pré-processados de alguma forma, conservando ainda colunas categóricas. A base de dados foi extraída do site <https://www.kaggle.com/> e pode ser encontrada [aqui](#).

1.3. Contexto do problema:

A Síndrome dos Ovários Policísticos (que recentemente teve seu nome alterado para Síndrome ovariana metabólica Poliendócrina) é um distúrbio hormonal comum, que estima-se que afete entre 10% e 13% das mulheres em idade reprodutiva em todo o mundo, sendo mais comum entre as mulheres com idade entre 30 e 40 anos.

Doença crônica, embora não grave, pode dificultar a capacidade da mulher de engravidar devido aos desequilíbrios hormonais que afetam a ovulação. Por isso, quanto mais cedo a condição foi diagnosticada, melhor conduzido pode ser o tratamento dos sintomas e mais chances a mulher tem de engravidar espontaneamente, caso assim o deseje.

2. DESCRIÇÃO DO DATASET

2.1. Dimensão dos dados:

O dataset escolhido tem um total de 684 amostras, coletadas da seguinte forma:

- 550 amostras coletadas a partir de datasets previamente publicados;
- 134 amostras coletadas a partir de um questionário online com respostas validadas e padronizadas.

No total, o dataset contém 33 colunas, sendo uma coluna a da variável-alvo e as demais 32 colunas de variáveis preditoras.

2.2. Tipos das variáveis:

A variável-alvo ('PCOS') já estava convertida para *integer* com os binários 0 e 1, sendo 0, não diagnosticada com SOP, e 1, diagnosticada com SOP.

Das 32 variáveis preditoras, somente **3** são variáveis **numéricas contínuas**, enquanto as demais **29** são **variáveis categóricas**. A maioria das variáveis categóricas são binárias do tipo "Sim" ou "Não" ("Yes"/"No"), porém, devido ao grande número de respostas do tipo "Eu não sei" ("I do not know"), as 8 colunas com esse tipo de valor acabaram sendo classificadas como não binárias, porque a categoria "I do not know" entrou como uma terceira categoria de dados ausentes ("Missing").

2.3. Distribuição da variável-alvo:

A base de dados mostrou-se desbalanceada para a variável-alvo, com mais de 75% das amostras correspondendo a pacientes diagnosticadas com a SOP, o contrário do que seria esperado, uma vez que, na população geral, a predominância é de mulheres não diagnosticadas com a doença.

3. ANÁLISE EXPLORATÓRIA DOS DADOS (EDA)

3.1. Primeiro contato:

A etapa da análise exploratória dos dados começou com uma inspeção simples no dataset: após observação das primeiras linhas da base por meio do método `.head()` da biblioteca Pandas, tive o primeiro contato com as colunas e seus respectivos tipos de dados. Por meio do método `.shape`, obtive a quantidade total de amostras e colunas do dataset e, na sequência, o método `.info()` me mostrou cada coluna com seu respectivo tipo de dado e quantidade de valores não nulos.

3.2. Dados ausentes:

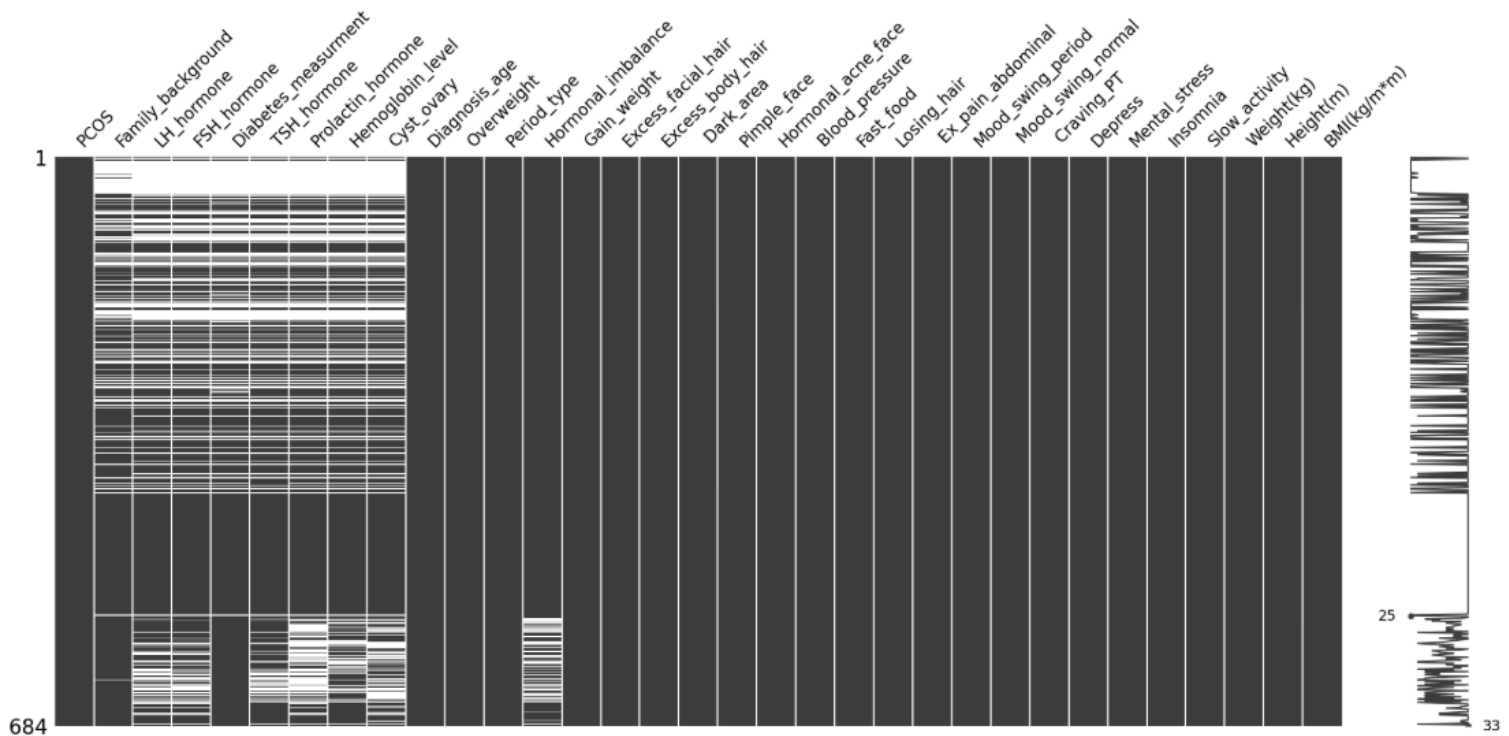
Embora os dados ausentes não estivessem explícitos no dataset (com células vazias ou com NaN, por exemplo), 8 das 32 colunas preditoras continham a classe/valor “I do not know” ou a sua variação “i do not know”, o que equivale à falta de dados onde as amostras têm esse valor. Uma nona coluna também, provavelmente por erro de digitação (*typo*), continha uma classe de valores que não se encaixa no perfil categórico da coluna (a coluna com dados binários - “Yes” ou “No” - tinha algumas amostras com o número 2), que precisou também ser considerada como dado ausente.

Abaixo, estão listadas todas as colunas que apresentaram dados ausentes seguidas do número de amostras sem dado para a coluna:

- *Family_background*: 148;
- *LH_hormone*: 221;
- *FSH_hormone*: 219;
- *Diabetes_mearurement*: 179;
- *TSH_hormone*: 201;
- *Prolactin_hormone*: 252;
- *Hemoglobin_level*: 213;
- *Cyst_ovary*: 235;
- *Hormonal_imbalance*: 50.

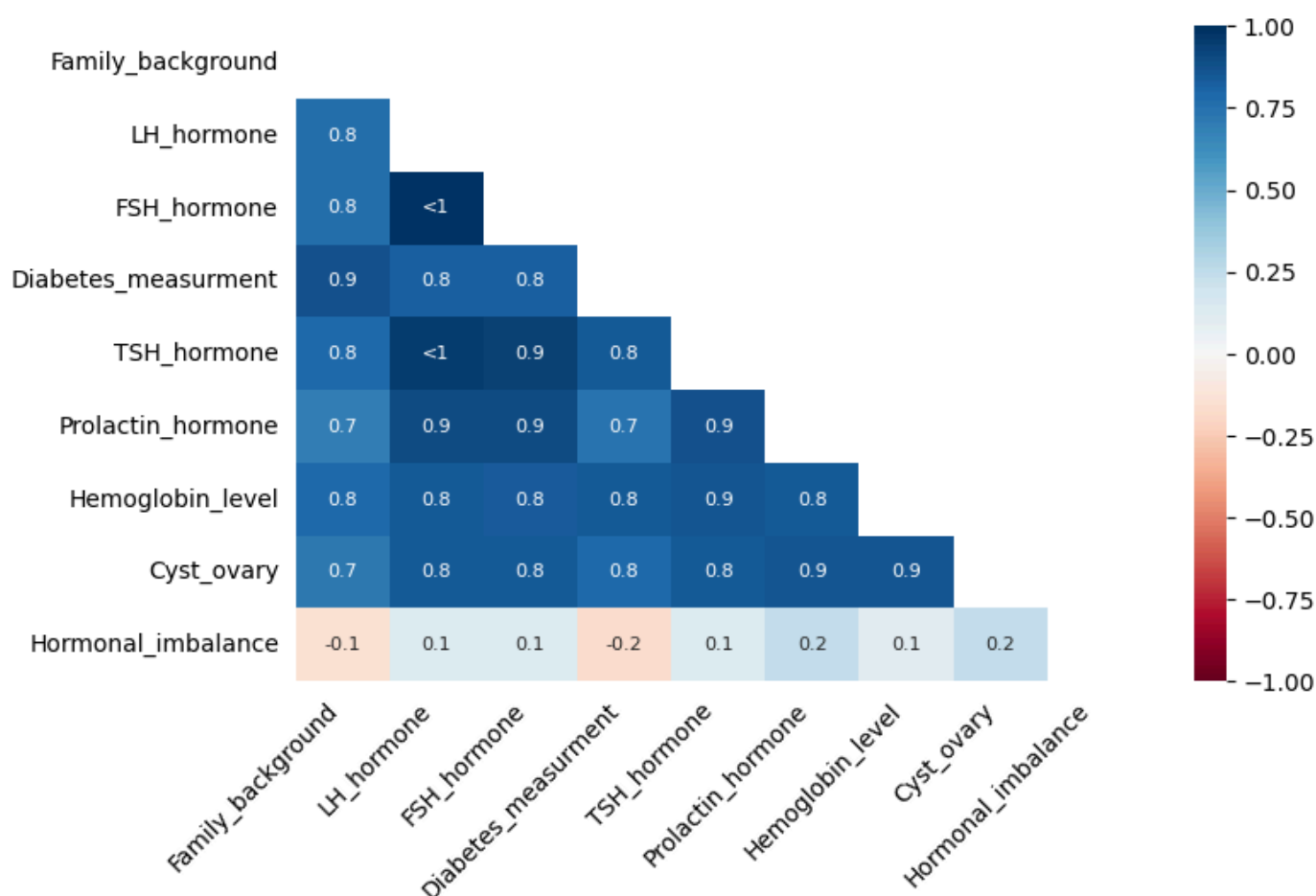
Com exceção da variável de histórico familiar (“*Family_background*”), podemos notar que todas as demais referem-se a informações laboratoriais a que algumas pacientes podem não ter tido acesso por falta de solicitação médica, por exemplo.

Ao usar a biblioteca Missingno para analisar mais de perto esses dados faltantes (temporariamente convertidos para NaN para poderem ser capturados pelos métodos da biblioteca), confirmamos a hipótese de que os dados ausentes nessas colunas categóricas “andam juntos”, ou seja, normalmente as amostras que não têm o dado para uma das colunas são as mesmas que não têm o dado para as demais, conforme nos mostra o gráfico abaixo obtido por meio do método `.matrix()`:



A variável “*Hormonal_imbalance*” foi a única que não seguiu tanto a mesma tendência das demais, concentrando toda a ausência de dados na parte final do dataset. Esta é a variável binária que continha valores numéricos (*integer 2*) “infiltrados” nas amostras. A concentração dessa classe de valor refuta um pouco a hipótese de que se trata de um erro de digitação, pois, para casos de *typo*, seria esperado que as amostras estivessem mais espalhadas pela base de dados.

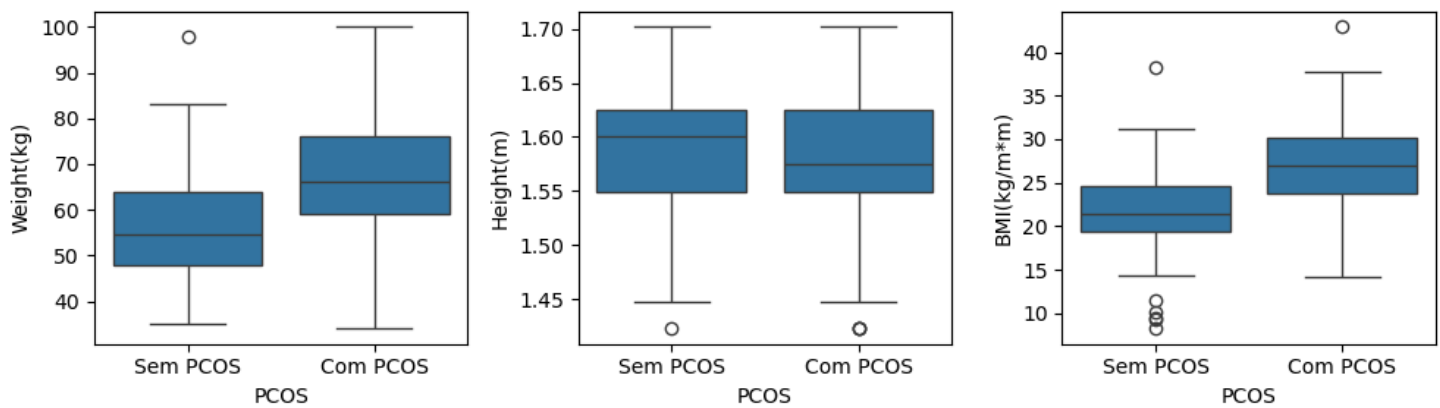
A presença de uma forte correlação entre a ausência dos dados nas 8 primeiras colunas categóricas pode ser confirmada pelo método `.heatmap()` da biblioteca missingno, conforme demonstrado abaixo:



Correlações superiores a 0.7 entre duas variáveis indicam que é alta a probabilidade de haver dado válido para a amostra em uma das variáveis quando há para a outra. O mapa de calor também evidencia o distanciamento das 8 primeiras colunas (em que os dados faltantes estavam explicitados com o valor “*I do not know*”) em relação à coluna “*Hormonal_imbalance*”, cujos dados ausentes estavam representados pelo número 2.

Uma última análise dos dados ausentes nas variáveis categóricas da base indicaram que, entre as mulheres **não** diagnosticadas com SOP, o **percentual** de dados ausentes foi significativamente maior do que nas mulheres que receberam o diagnóstico.

Nas variáveis contínuas, por outro lado, a quantidade de dados nulos foi substancialmente menor. Num primeiro momento, esses dados faltantes também não foram identificados pelo método tradicional (.isnull()) porque todas as amostras tinham um valor considerado válido. Porém, ao plotar os gráficos boxplot para analisar de forma visual as principais métricas estatísticas dessas variáveis, identificamos que as colunas de altura (“*Height(m)*”) e IMC (“*BMI(kg/m*m)*”) continham valores zerados (0.0), conforme fica evidente nos gráficos abaixo:



Confirmei os valores zerados e usei o método `.sum()` para verificar a quantidade de valores, já que o boxplot sobrepõe os marcadores nas amostras com o mesmo valor. Tanto a variável de altura quanto a variável de IMC tinham somente 2 valores nulos cada, o que levantou a hipótese de que se tratavam das mesmas 2 amostras, porque o cálculo do IMC depende do valor da altura, e um valor de altura nulo consequentemente vai gerar um valor de IMC igualmente nulo. Uma inspeção rápida na base confirmou a hipótese.

3.3. Distribuição das variáveis numéricas:

Para começar a análise das variáveis numéricas, eu usei os métodos `.mean()` e `.median()` para ver as médias e medianas de cada uma das variáveis para os grupos de pacientes diagnosticadas e não diagnosticadas com SOP. Em vez de olhar para todas as métricas estatísticas com o método `.describe()`, preferi focar apenas naqueles que me trariam algum *insight* importante para a análise, motivo pelo qual eu preferi ver as médias e medianas por classe da variável-alvo, em vez de olhar para o total do dataset.

Como eu já suspeitava por conhecer um pouco sobre a Síndrome dos Ovários Policísticos, pode ser inferida uma diferença no peso corporal de mulheres com e sem a SOP. Avaliando em conjunto com o IMC, podemos afirmar que há mais mulheres com sobrepeso entre as diagnosticadas com a doença do que entre as não diagnosticadas. Para a variável altura, não foram encontradas diferenças significativas.

MÉDIA

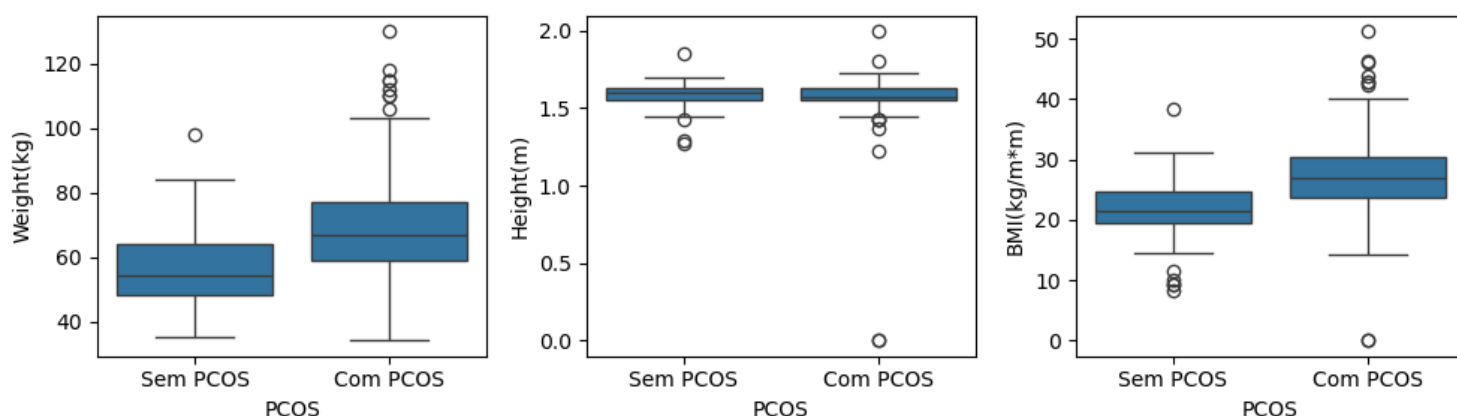
	Weight(kg)	Height(m)	BMI(kg/m²)
Sem PCOS	56.079755	1.585316	21.910429
Com PCOS	68.174088	1.574525	27.06737

MEDIANA

	Weight(kg)	Height(m)	BMI(kg/m*m)
Sem PCOS	54.0	1.6002	21.5
Com PCOS	67.0	1.5748	27.0

Pela proximidade entre os valores das médias e das medianas, podemos supor que as variáveis têm poucos outliers ou que eles não são tão expressivos para “puxar” as médias para muito acima ou muito abaixo dos valores das medianas.

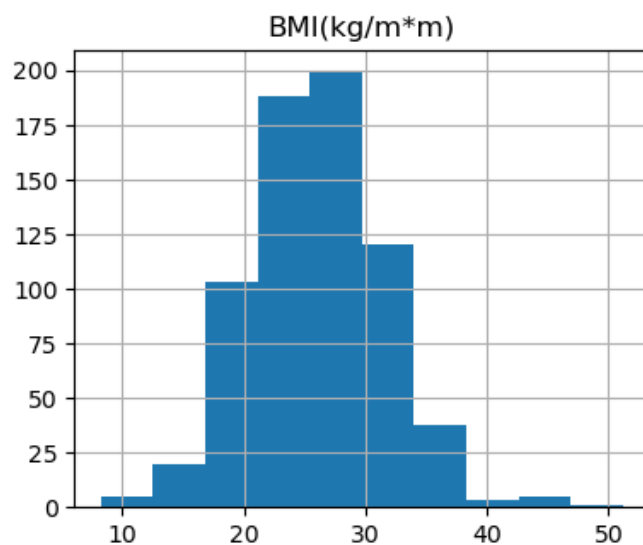
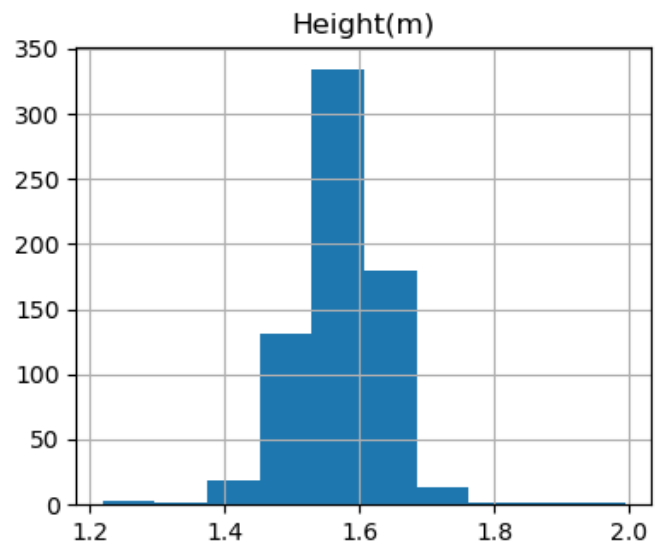
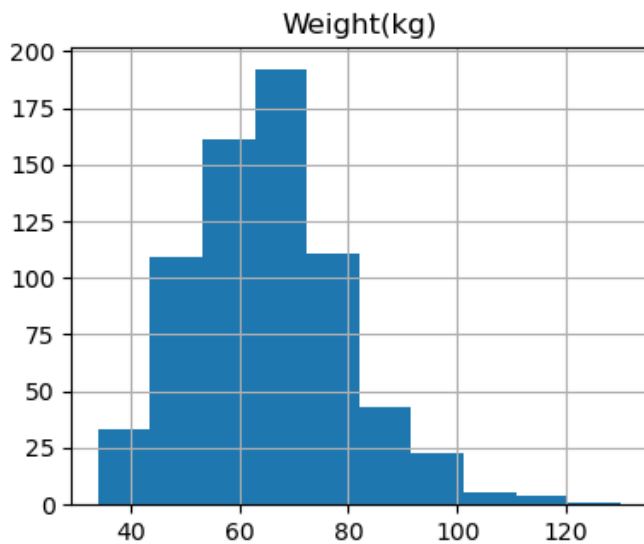
Ainda mantendo as variáveis agrupadas por cada classe da variável-alvo, optei por começar minha análise gráfica com gráficos do tipo **boxplot**, porque eles nos trazem todas as principais métricas estatísticas de maneira bem visual. Confrontando cada variável com a variável-alvo, as diferenças entre as duas classes para as variáveis de peso e IMC ficou ainda mais evidente:



Pelos gráficos, pudemos não apenas confirmar que mulheres com SOP realmente têm métricas de peso e IMC mais elevadas que as mulheres sem SOP, mas também pudemos notar uma concentração de outliers acima dos valores médios, indicando que mulheres com a doença podem chegar a desenvolver problemas de obesidade.

Outra informação que estes gráficos nos trouxeram foi a identificação dos dados ausentes nas colunas “*Height(m)*” e “*BMI(kg/m*m)*”, que discutimos no tópico anterior. Vamos ver como foi feito o tratamento desses dados ausentes na parte do pré-processamento dos dados.

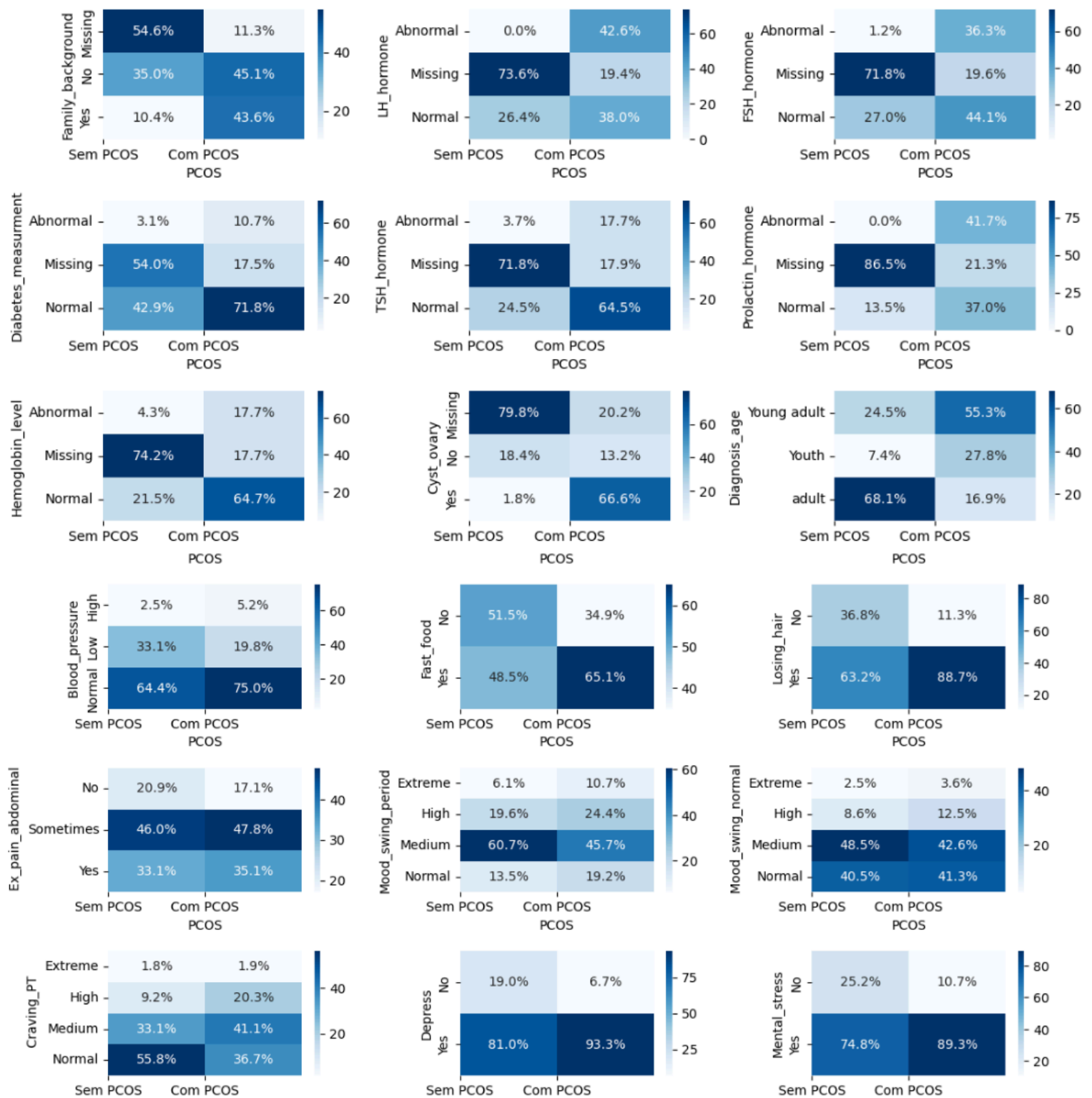
Por fim, a fim de ver a distribuição dos dados contínuos, plotei também os histogramas dessas variáveis:

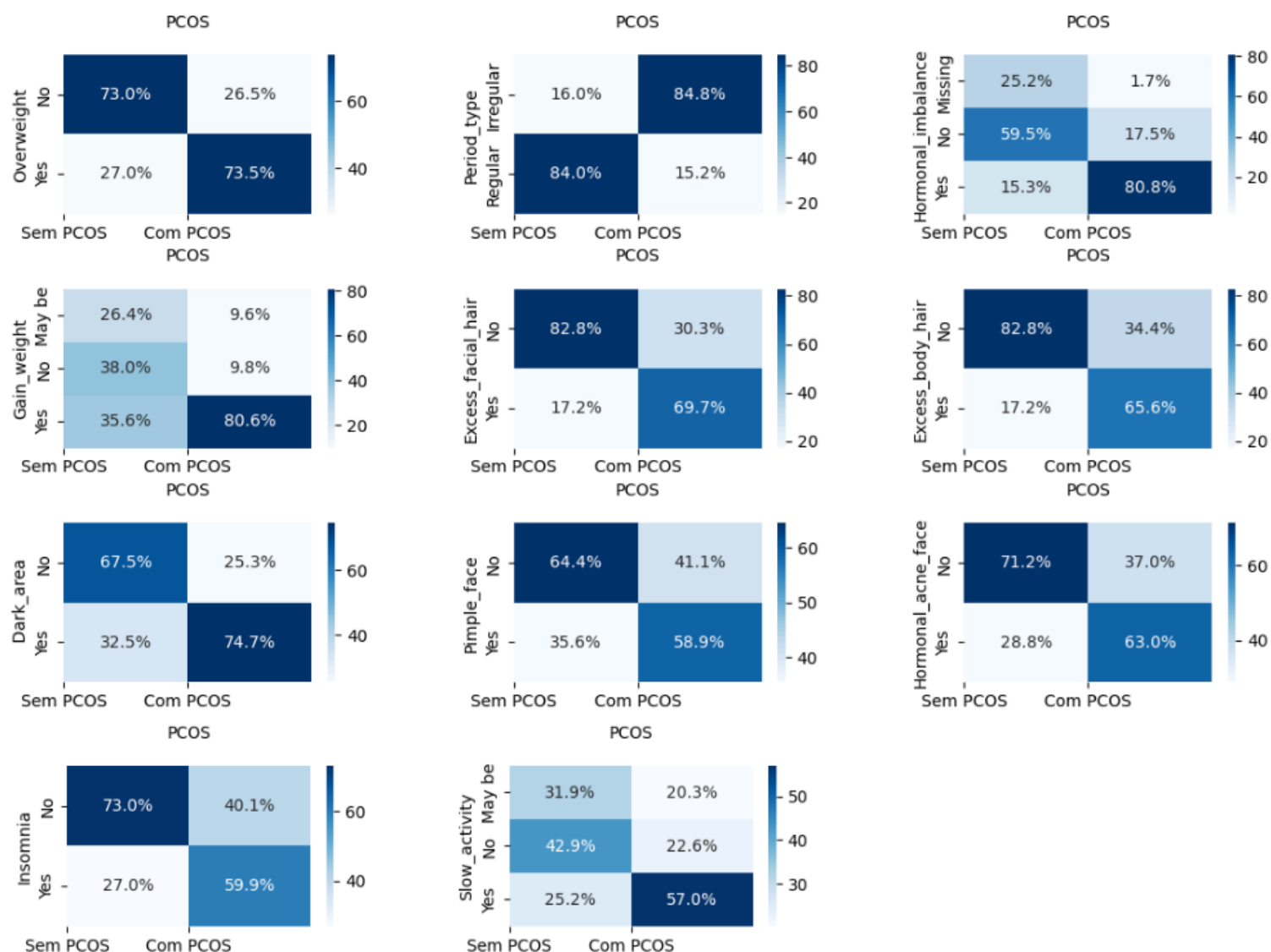


Das 3 variáveis contínuas, a altura foi a que mais se aproximou de uma distribuição normal, com uma curva que lembra o formato de sino da curva gaussiana. Porém, ao aplicar o Teste de Shapiro-Wilk, com o método `.shapiro()` da biblioteca SciPy, foi constatado que nenhuma delas de fato tem uma distribuição normal.

3.4. Análise das variáveis categóricas:

Analisar variáveis binárias contra variáveis binárias é uma tarefa ingrata. Não é possível plotarmos gráficos de distribuição ou de dispersão que nos tragam insights relevantes e, mesmo depois de codificadas, a correlação entre elas não é captada de forma eficaz. Decidi, então, fazer a análise criando **tabelas cruzadas** que confrontassem cada variável categórica com a variável-alvo e plotando isso em pequenos mapas de calor na sequência. O resultado foi o abaixo:





Da observação dessas tabelas, foi possível tirar os seguintes insights:

- Entre as mulheres **sem SOP** que fizeram exames dos níveis do hormônio LH, **nenhuma** apresentou níveis anormais do hormônio, enquanto **quase a metade (42,8%) das mulheres com SOP** apresentou anomalias neste mesmo exame. Da mesma forma, apenas **1,2%** das mulheres **sem SOP** que fizeram exames de hormônio FSH tiveram níveis anormais do exame, contra mais de **um terço (36,7%) das mulheres com SOP**. De maneira geral, o percentual de mulheres sem SOP com alguma anormalidade nos exames hormonais é bem mais baixo do que o percentual das mulheres com SOP, mas é difícil tirar conclusões “absolutas”, porque há muita ausência de dados, principalmente entre as mulheres sem SOP;
- Outro dado que chamou a atenção é o de **cisto no ovário**, presente em **dois terços (66,9%) das mulheres diagnosticadas com SOP**, mas em apenas **1,9%**

das mulheres **sem SOP**, mas novamente a comparação fica comprometida pela falta de dados muito elevada entre as mulheres sem a condição;

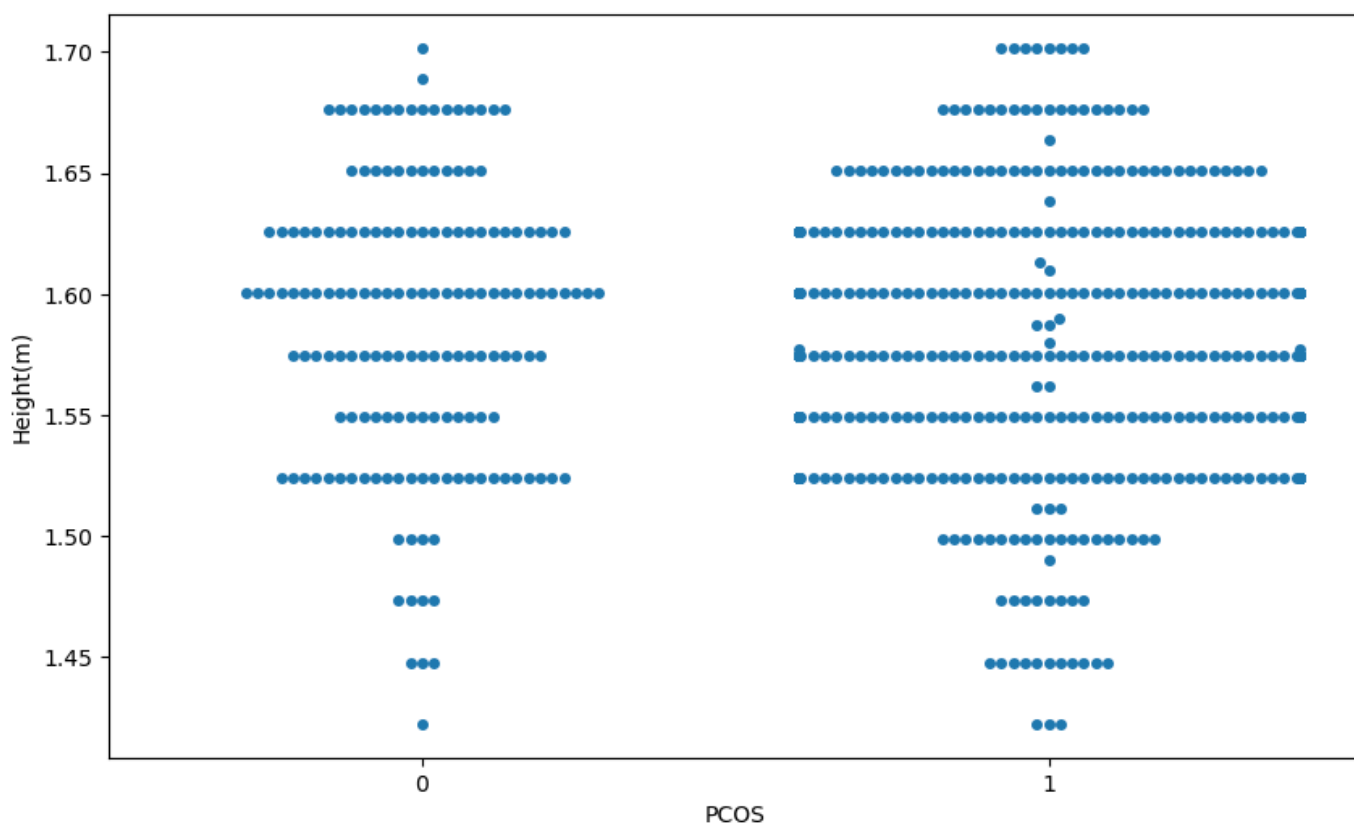
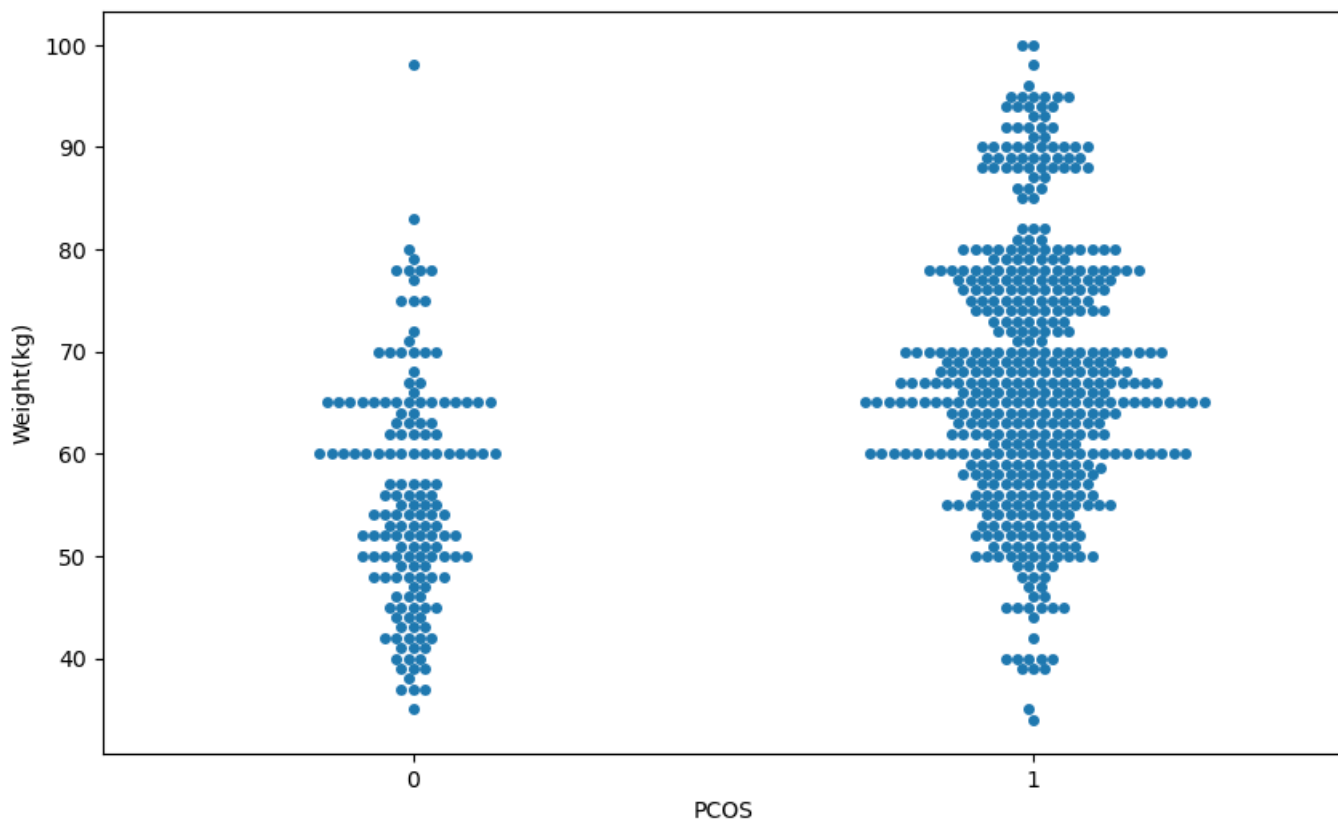
- Nas categorias sem o problema de ausência de dados, fica mais fácil enxergar correlações provavelmente verdadeiras. O **sobrepeso** e a **regularidade do ciclo menstrual** são duas categorias em que a diferença entre mulheres com SOP e sem SOP fica gritante. São **73,2% de mulheres com SOP** que têm sobrepeso, contra somente **26,9% das mulheres sem SOP**. Enquanto **85% das mulheres com SOP** têm ciclos menstruais irregulares, somente **16,2% das mulheres sem SOP** sofrem com o mesmo problema;
- Outras categorias em que a diferença é nítida: **ganho de peso (similar à categoria de sobrepeso), excesso de pelos faciais e corporais, áreas escurecidas, espinhas e acne hormonal, insônia e atividade mais lenta.**

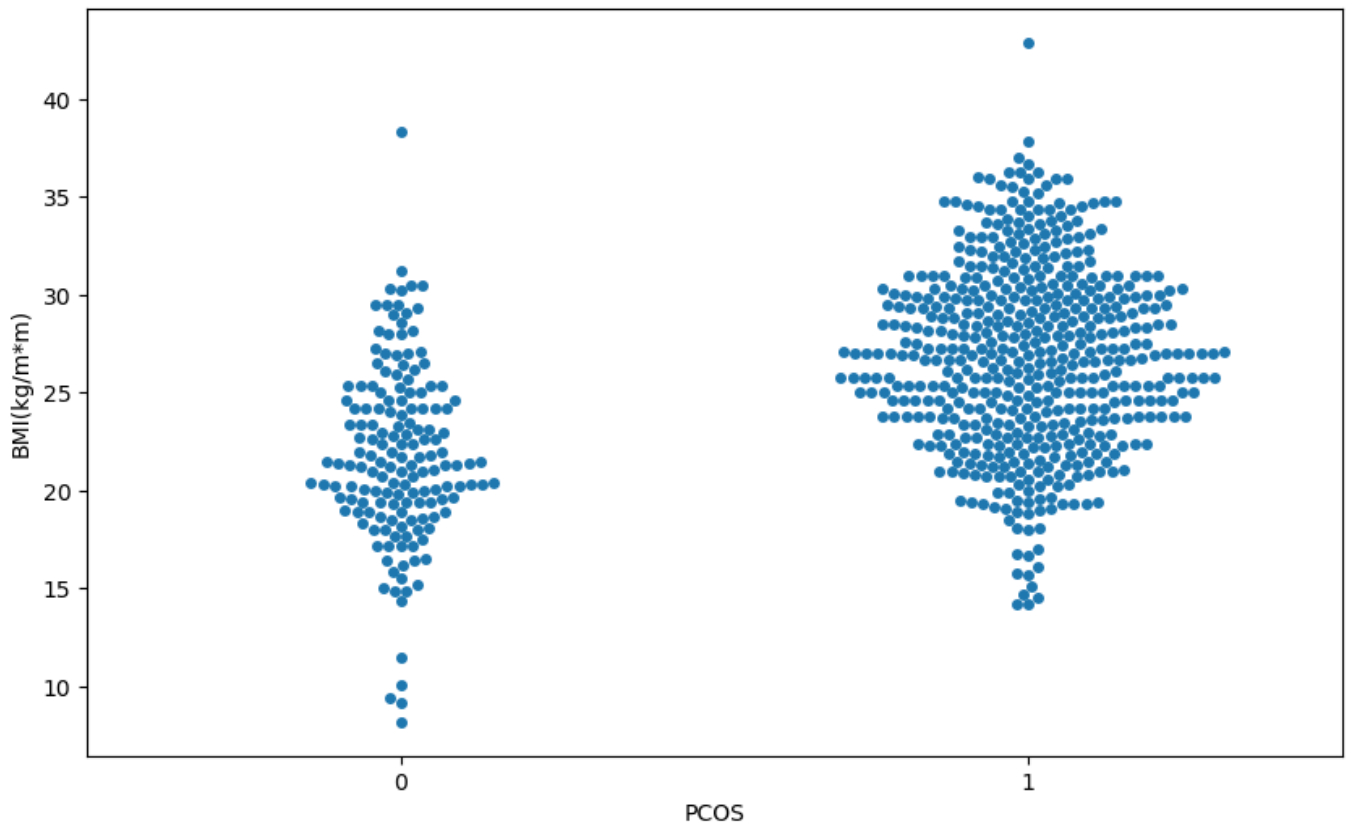
3.5. Identificação dos outliers:

Neste dataset, outliers relevantes só foram encontrados nas nossas 3 variáveis de dados contínuos, porque as variáveis categóricas têm poucas classes (grande parte binária) e mesmo classes com frequência mais baixa (como pressão alta, na variável de pressão sanguínea, ou mudança extrema de humor, na variável de variações de humor fora do período menstrual) não têm tanto impacto para a nossa análise.

Os gráficos do tipo boxplot que plotamos nesta etapa de análise exploratória deixaram os outliers das categorias peso, altura e IMC bem evidentes para nós. Nesta primeira análise, pudemos ver que a variável peso só tinha outliers para cima dos valores médios, enquanto altura e IMC apresentaram outliers tanto para cima quanto para baixo da maioria dos dados.

Embora o boxplot seja excelente para nos indicar os outliers, nós não conseguimos ter uma visão da frequência de amostras com o mesmo valor, porque o gráfico sobrepõe os marcadores. Por esse motivo, plotei também gráficos do tipo swarmplot que fazem uma distribuição lado a lado de amostras com valores iguais:





Os *swarm plots* nos deram uma melhor visibilidade da quantidade de outliers em cada coluna. A variável altura, por exemplo, parece ter muitos outliers abaixo da concentração dos dados, enquanto, para o peso, identificamos um bloco considerável de amostras acima dos 80 quilos (não chegando a configurar outliers pela quantidade de amostras nessa faixa), com poucas amostras destoando bastante (acima dos 100 quilos) e predominância desses outliers entre as mulheres diagnosticadas com SOP.

Pensando um pouco à frente, sobre como faria o escalonamento das variáveis na etapa de pré-processamento dos dados, optei por remover os outliers mais extremos para que eles não impactassem uma possível normalização dos dados, usando o algoritmo *MinMax Scaler*.

3.6. Principais achados da análise exploratória dos dados:

Ao investigar e analisar a base de dados selecionada, pude chegar às seguintes conclusões:

- A base de dados tem poucas amostras, mas um número de colunas (variáveis) elevado em relação ao seu tamanho. A dimensionalidade dos dados pode ser considerada média, com potencial de aumentar consideravelmente durante a etapa de codificação das variáveis categóricas. Isso nos leva a considerar a aplicação de

uma técnica para a redução da dimensionalidade dos dados, principalmente se formos usar o algoritmo K-nearest-neighbors (KNN);

- Há um desbalanceamento considerável na variável-alvo. Trata-se de uma variável binária (doente ou não doente), em que a classe “doente” é bem predominante (76,2% dos dados). Isso nos indica a importância de aplicar o hiperparâmetro “stratify” na hora da separação da nossa base em conjuntos de treino e teste, além de usar hiperparâmetros específicos dos algoritmos classificadores para amenizar essa diferença;
- Ausência de dados elevada em 8 das 32 variáveis preditoras, a maioria delas relativa a dados clínicos. Essa ausência está concentrada no mesmo grupo de amostras, indicando possivelmente falta de pedido médico para a realização de exames, além de ser percentualmente predominante entre as mulheres sem diagnóstico de SOP. Outras 3 colunas também apresentaram dados ausentes, mas em quantidades menos significativas (50 para a coluna de desbalanceamento hormonal e apenas 2 amostras para altura e IMC). Pela expressividade da representação desses dados em relação ao total da amostra, será necessário lidar com a sua falta como uma categoria extra, já que removê-los do dataset teria grande impacto na quantidade disponível de dados, e substituí-los pelo valor da moda seria leviano, pois certamente enviesaria os resultados;
- Pela análise dos dados contínuos, foi possível notar que, apesar de haver alguns outliers, eles não foram expressivos o suficiente para alterar muito a média dos dados que, para as 3 variáveis numéricas, ficou muito próxima ao valor da mediana (percentil 50%). Olhando para a forma como os dados estão distribuídos entre mulheres com e sem o diagnóstico de SOP, encontramos uma concentração de mulheres com IMC acima de 30.0 (obesidade) entre as mulheres com a doença que também concentraram os outliers positivos para a variável peso. Isso já indica uma possível correlação entre a Síndrome dos Ovários Policísticos e o sobrepeso/obesidade;
- Já para a variável de altura, não foram encontradas diferenças significativas entre os grupos de mulheres com e sem a condição médica. Isso indica a pouca relevância da variável para a nossa classificação;
- Em relação às variáveis categóricas, podemos inferir que a Síndrome dos Ovários Policísticos parece estar relacionada à secreção de hormônios importantes, como o

TSH e a prolactina, mas especialmente de hormônios gonadotróficos (LH e FSH); foi encontrado um percentual muito maior de mulheres com níveis anormais desses hormônios entre aquelas diagnosticadas com SOP. Mais de 80% das mulheres com a doença reportaram ter níveis hormonais desbalanceados, o que é uma das causas da Síndrome dos Ovários Policísticos, porque pode fazer com que a ovulação não ocorra de forma regular, o que explica também a alta taxa (84,8%) de mulheres com ciclo menstrual irregular entre as mulheres com SOP;

- Outros achados são maior predominância de sobrepeso/obesidade, maior presença de excesso de pelos (corporais e/ou faciais), maior incidência de acne e de problemas para dormir (insônia) e/ou de falta de energia (atividade lenta) entre as mulheres com o diagnóstico. Tudo isso aponta de fato para problemas hormonais que caracterizam a Síndrome dos Ovários Policísticos.

4. PRÉ-PROCESSAMENTO DOS DADOS

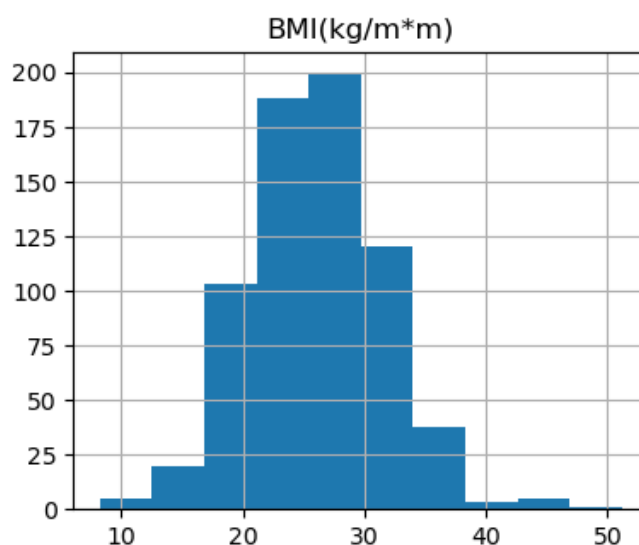
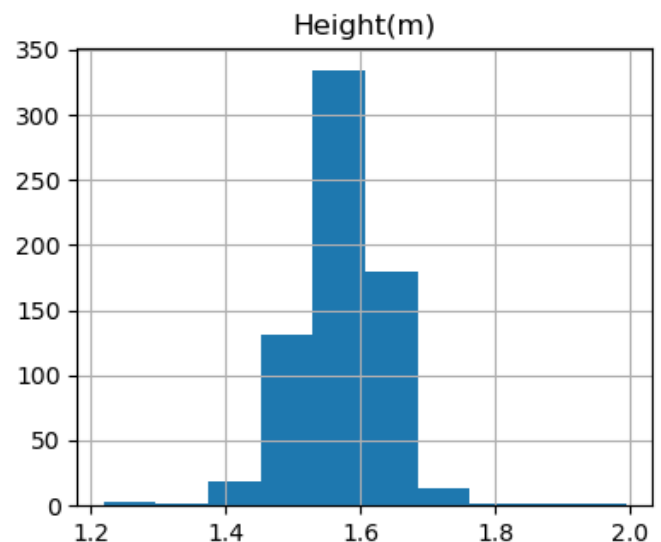
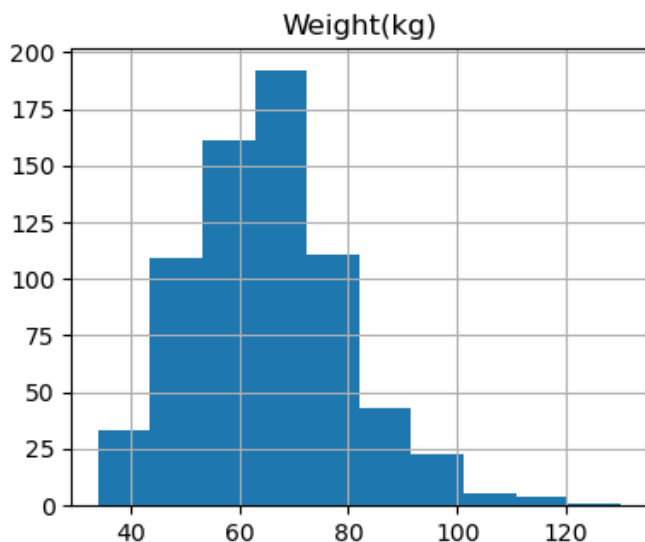
4.1. Tratamento dos dados ausentes:

De maneira geral, dados ausentes em variáveis categóricas podem ser preenchidos com a moda dos dados (classe mais comum na coluna) ou até mesmo removidos do dataset, porém, no presente caso, a grande quantidade de dados ausentes representava um percentual importante do total das amostras, o que nos impossibilitou de resolver a questão de forma tão simplista. Substituir os dados ausentes pelo valor da moda iria inserir um grande viés para o treinamento do algoritmo, enquanto excluir as amostras em uma base de tamanho já reduzido também prejudicaria a etapa de treinamento.

Com isso em mente, optei por manter os dados com as classes denotadoras de ausência de informação, apenas fazendo uma padronização para que todos seguissem a mesma classe de valor: "*Missing*".

Para os dados ausentes nas variáveis categóricas, entretanto, o tratamento foi outro. Como havia apenas duas amostras com valores nulos para as colunas de altura e IMC, optei por substituir esses valores pela média ou mediana dos dados na coluna de altura enquanto, para a coluna de IMC, eu seguiria com o cálculo desta métrica ($\text{peso}/\text{altura}^2$) a partir dos valores preenchidos para as alturas.

Para decidir se eu usaria a média ou a mediana para substituição das alturas nulas, eu aproveitei para plotar os histogramas das variáveis contínuas, visando a entender se a distribuição dos dados para a variável “*Height(m)*” seguia uma distribuição normal ou não. **Isso foi feito ainda na etapa de análise exploratória.**



Apesar de o histograma da variável de altura ter um formato que lembre a curva gaussiana, o teste de shapiro, aplicado por meio do método `.shapiro()` da biblioteca SciPy, nos mostrou que a distribuição da variável “*Height(m)*” **não é uma distribuição normal** e, portanto, embora a média e a mediana sejam números próximos, eu optei por fazer a substituição dos 2 dados nulos pela mediana da altura, que é a recomendação para distribuições que não seguem a curva de Gauss.

Feita a substituição dos dados ausentes de altura, os dados ausentes para o IMC puderam ser calculados com os novos valores preenchidos na coluna “*Height(m)*”.

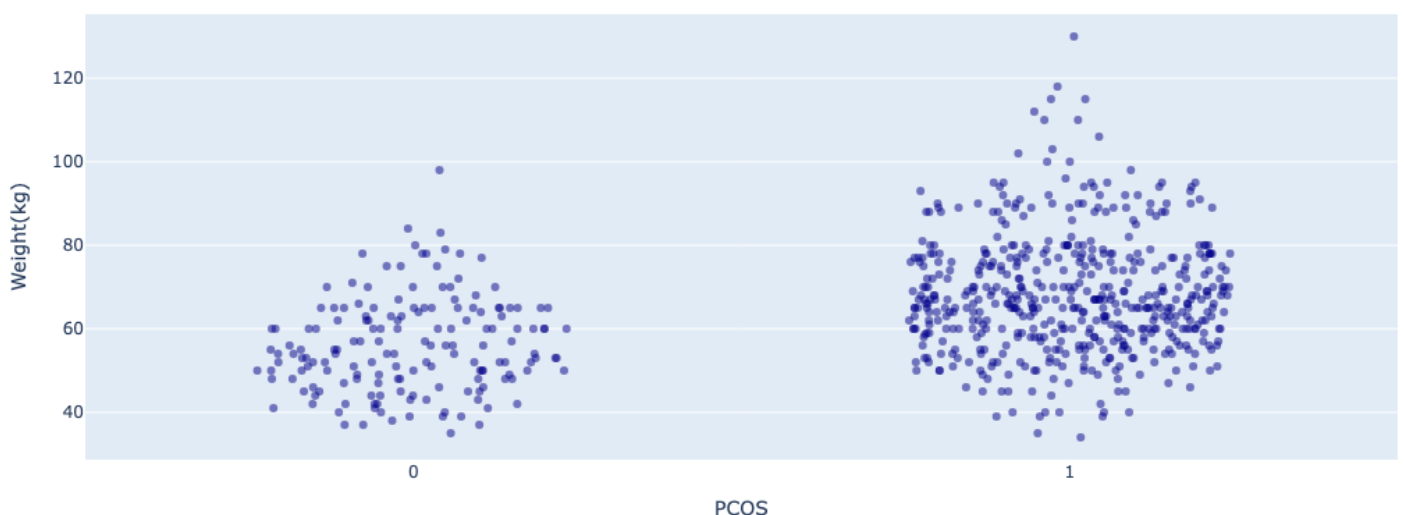
4.2. Tratamento dos outliers:

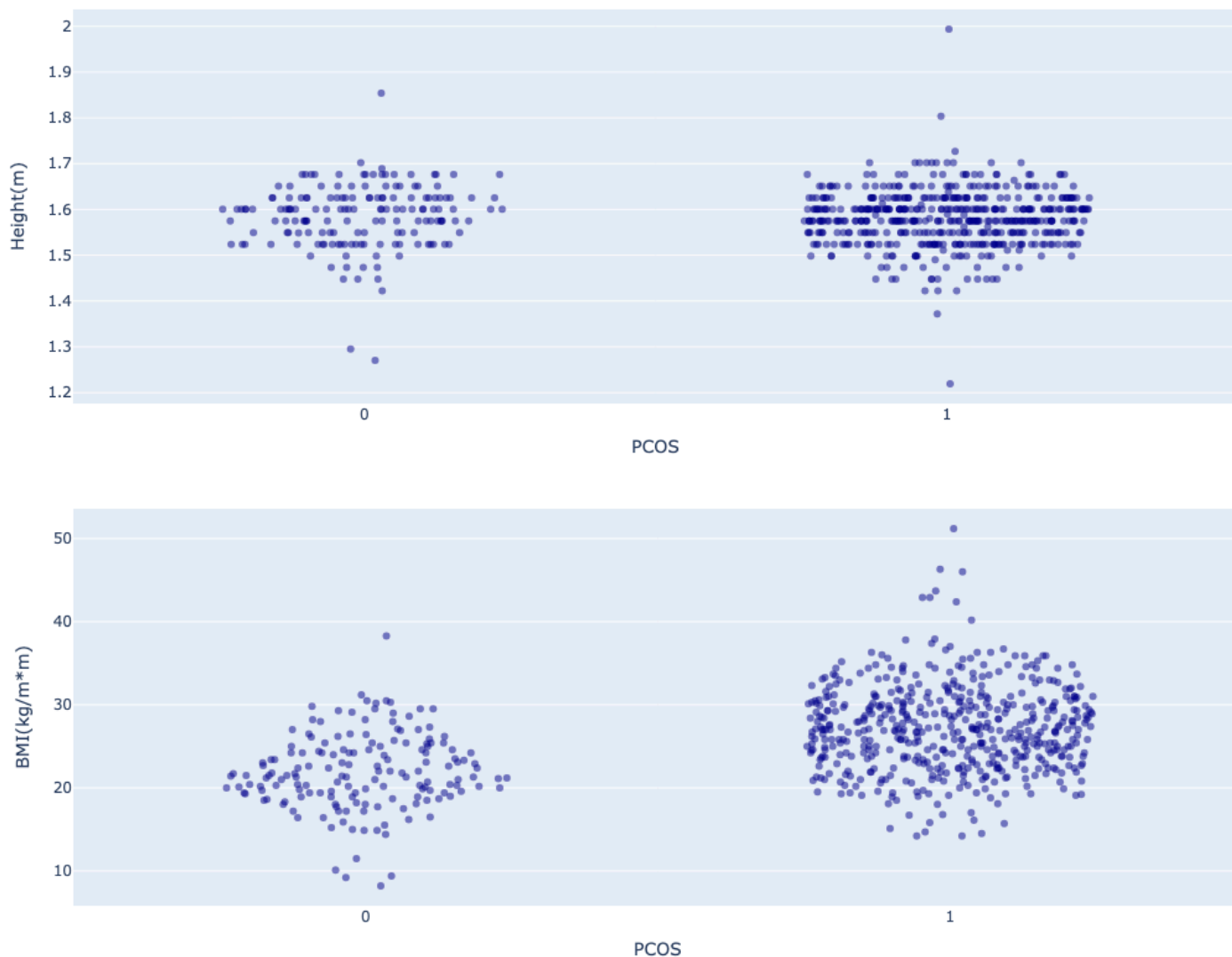
O dataset escolhido para este estudo não apresentou um número muito grande de outliers para as nossas variáveis contínuas, não chegando a ultrapassar 4% dos dados aproximadamente. Pelo que os gráficos de boxplot nos mostraram, a variável de peso (“*Weight(kg)*”) somente tinha alguns outliers acima da média dos valores, enquanto, para as variáveis de altura (“*Height(m)*”) e IMC (“*BMI(kg/m*m)*”), havia alguns outliers acima e outros abaixo da concentração da média dos valores.

A princípio, eu poderia ter deixado os outliers na base, mas, durante a etapa de pré-processamento dos dados, vi-me inclinada a usar o processo de normalização (*Min Max Scaler*) para escalonar esses dados numa escala entre 0 e 1 e, durante esse processo, outliers muito discrepantes podem acabar achatando demais o restante dos dados em um intervalo muito pequeno que não seja representativo da variância real dos dados. Por esse motivo, optei por remover boa parte dos outliers da base.

Para entender quais valores seriam minhas faixas de corte para remoção das amostras do dataset, eu precisei plotar outro gráfico que me trouxesse os valores dos pontos destoantes do conjunto. Com a biblioteca plotly, seria possível obter um gráfico similar ao swarmplot que me permitiria visualizar o valor exato de cada ponto do gráfico no hover do mouse. Como abaixo esses gráficos estão representados como imagens estáticas, eles não exibem os valores, mas, no notebook deste trabalho, é possível visualizá-los passando o mouse por cima de cada ponto.

Identificação Visual de Outliers





A partir da análise dos gráficos boxplot em conjunto com os gráficos acima, optei por remover todas as amostras que tivessem um valor acima de 100kg para a variável de peso, ou ainda abaixo de 1.44 ou acima de 1.73 para a variável de altura.

Fiz a remoção desses valores por meio do método `.drop()` e, na sequência, plotei novamente os boxplot para verificar se tinha eliminado a totalidade ou a quase totalidade dos outliers. A surpresa, no entanto, veio no gráfico do IMC (" $BMI(kg/m^2)$ "), porque, mesmo tendo removido os valores extremos de peso e altura, ainda restaram muitos outliers para baixo do valor médio na coluna do IMC, o que não deveria acontecer.

Já com suspeitas de que alguns valores desta coluna haviam sido calculados erroneamente, mesmo estando descrito no Kaggle que os valores desta coluna tinham sido calculados pela divisão do peso pelo quadrado da altura, resolvi dar uma olhada nas amostras com valores mais baixos de IMC. Descobri que realmente as cinco amostras com os menores valores de IMC estavam com o cálculo errado; então optei por recalcular

os valores desta coluna para toda a base (respeitando o cálculo que estava indicado no Kaggle) e, assim, os gráficos boxplot nos mostraram que foram eliminados quase 100% dos outliers.

4.3. Separação da base:

Conforme foi solicitado na descrição do tech challenge, fiz a separação da base nos conjuntos de treino e teste, embora minha primeira escolha fosse não separar os dados e aplicar a técnica de validação cruzada.

A técnica de validação cruzada é muito boa para datasets pequenos, porque nos permite usar a totalidade dos dados tanto na etapa de treino quanto na etapa de teste, criando n instâncias diferentes do algoritmo escolhido e fazendo n etapas de treinos e testes com conjuntos diferentes randômicos a cada vez.

Mas, por solicitação do desafio, fiz a separação do dataset nos dois conjuntos durante a etapa de pré-processamento dos dados. Optei por limpar os valores nulos e os outliers da base antes da separação, porque separar os conjuntos de treino e teste usando o *train_test_split* exige também separar a nossa variável-alvo das variáveis preditoras, e eu não poderia remover uma amostra do conjunto X (variáveis preditoras) sem removê-la também do y (variável-alvo).

Desta forma, logo após o tratamento dos nulos e outliers, separei a coluna “PCOS” do restante da base, pois ela é nossa variável de interesse (alvo). Na sequência, prossegui usando o algoritmo *train_test_split* para dividir os conjuntos de treino e teste, usando o hiperparâmetro *stratify*, para garantir que ambos os conjuntos manteriam a mesma proporção de mulheres com e sem o diagnóstico de SOP (PCOS), uma vez que o dataset é bastante desbalanceado, com predominância de mulheres com a doença.

4.4. Codificação das variáveis categóricas:

No dataset selecionado, as variáveis categóricas podem ser divididas em dois grupos: as variáveis binárias (apenas duas classes) e as variáveis com múltiplas classes, (que, neste dataset, tinham 3 ou 4 classes diferentes). Algumas das variáveis binárias (de respostas do tipo “Yes” ou “No”), no entanto, entraram como variáveis de múltiplas classes devido à terceira categoria de dados não informado (“Missing”).

Devido a essa diferença entre as variáveis categóricas, optei por usar dois algoritmos de codificação diferentes: para as variáveis binárias, o *Ordinal Encoder*, para as multiclases, o *One Hot Encoder*.

O *Ordinal Encoder* é um algoritmo de codificação de variáveis categóricas muito utilizado, mas a recomendação é usá-lo para codificar variáveis que mantêm algum tipo de hierarquização entre si, como, por exemplo, numeração de roupas (do PP ao GG). Isso acontece porque o *Ordinal Encoder* atribui números inteiros ordenados a cada variável (0, 1, 2, 3, 4...), o que pode ser lido como uma ordenação/hierarquização por alguns algoritmos de machine learning.

Apesar dessa recomendação, o *Ordinal Encoder* também funciona muito bem para a codificação de variáveis binárias, já que o resultado será apenas a transformação das duas classes em 0 e 1, o que é perfeitamente compreendido como *True* e *False* pelo computador, que é justamente o que eu estava buscando para a codificação dessas variáveis. Assim, definir que, para cada coluna binária, o “No” seria 0, enquanto o “Yes” seria 1 (no caso da coluna de regularidade do ciclo menstrual, ciclos regulares ficaram com o 0, denotando ausência de irregularidades, enquanto ciclos irregulares receberam o número 1).

Para as demais variáveis categóricas, optei pelo uso do algoritmo *One Hot Encoder*, que também é muito popular no meio de machine learning para codificar variáveis que não tenham muitas classes diferentes. No caso das nossas variáveis não binárias, a maioria delas tinha apenas 3 classes, enquanto algumas poderiam ter 4, o que são números pequenos, adequados para o trabalho com o *One Hot Encoder*.

O *One Hot Encoder* não é recomendado para codificar variáveis que podem ter muitas classes diferentes, porque cada classe gera uma nova coluna, o que aumenta de forma exponencial a dimensionalidade do dataset. No nosso caso, saímos de 32 colunas preditoras para 52, fazendo uso do hiperparâmetro “*drop='first'*” para conter um pouco o número extra de colunas.

Com a cardinalidade baixa, o uso do *One Hot Encoder* é mais indicado do que o uso de um outro algoritmo como o *Target Encoder*, por exemplo, porque este último aumentaria a complexidade de maneira desnecessária, introduzindo riscos de *overfitting* e de perda de informação para classes pouco recorrentes, enquanto o *One Hot Encoder* consegue criar uma representação mais fiel e sem precisar levar em conta os valores da coluna da variável-alvo.

4.5. Escalonamento das variáveis contínuas:

Esta talvez tenha sido a parte que me gerou mais dúvida, embora a aplicação do algoritmo de escalonamento em si seja simples.

Em um dataset como o selecionado, que contém tanto variáveis categóricas quanto variáveis contínuas, é difícil encontrar um consenso sobre a necessidade de escalonar ou não as variáveis categóricas após sua codificação.

No caso deste estudo, todas as variáveis categóricas foram codificadas para colunas contendo 0 ou 1. Para o escalonamento das variáveis contínuas, eu poderia usar a padronização - que costuma ser mais recomendada quando o dataset tem muitos outliers ou para uso com algoritmos baseados em distância, como o KNN e o SVM -, mas a padronização costuma deixar os dados em uma escala que varia entre -3 e +3 na maioria dos casos. Sem escalonar as variáveis categóricas usando a mesma técnica, os algoritmos atribuiriam maior importância aos dados contínuos, que teriam uma variância maior.

Como não há consenso sobre o escalonamento das variáveis categóricas codificadas e há certa preocupação com a perda da natureza discreta e intuitiva dos dados, optei por aplicar o escalonamento somente sobre as variáveis contínuas. Neste caso, então, preferi usar a normalização com o algoritmo *Min Max Scaler* porque eu poderia concentrar meus dados na mesma escala dos demais - entre 0 e 1.

Foi por esse motivo que, anteriormente, fizemos a remoção dos outliers, visando evitar que valores extremos achatassem os demais dados e prejudicassem a variância dos dados. Assim, aplicamos o *Min Max Scaler* com mais tranquilidade e garantimos que todas as nossas variáveis ficassem na mesma escala numérica.

5. REDUÇÃO DE DIMENSIONALIDADE

5.1. Por que aplicar o PCA:

A técnica do PCA - Análise de Componentes principais - é uma técnica usada para reduzir a dimensionalidade dos nossos dados. Nem sempre isso é necessário, mas, em datasets com um número muito grande de variáveis preditoras, o excesso de dados pode

ser prejudicial, confundir o algoritmo e levar, por exemplo, ao *overfitting*. Nem toda coluna que está no dataset realmente precisa fazer parte do grupo de variáveis preditoras.

O número de colunas do nosso dataset, mesmo após a codificação com o *One Hot Encoder*, não estava tão elevado, mas, quando pensamos em aplicar alguns algoritmos baseados em distância, como o KNN, um espaço multidimensional muito grande pode prejudicar o agrupamento dos dados.

Como uma etapa do pré-processamento, eu poderia também optar por incluir ou não a redução da dimensionalidade no meu pipeline de preparação dos dados e até mesmo testar o mesmo algoritmo com os dados com e sem o PCA aplicado. Por esse motivo, decidi que valia a pena já calcular o meu número de componentes principais e ter o PCA pronto para ser aplicado caso assim eu decidisse.

5.2. Análise da variância acumulada:

Numa análise de componentes principais, cada componente definido pelo algoritmo do PCA consegue representar um percentual do total de informação que o dataset traz para nós. Essa informação é obtida por meio da variância dos dados: quanto maior a variância de uma determinada variável preditora, mais informação essa variável é capaz de trazer para a nossa análise. Portanto, cada componente do PCA representa um percentual da variância total correspondente a 100% do dataset; este percentual é a taxa de variância que o componente consegue captar.

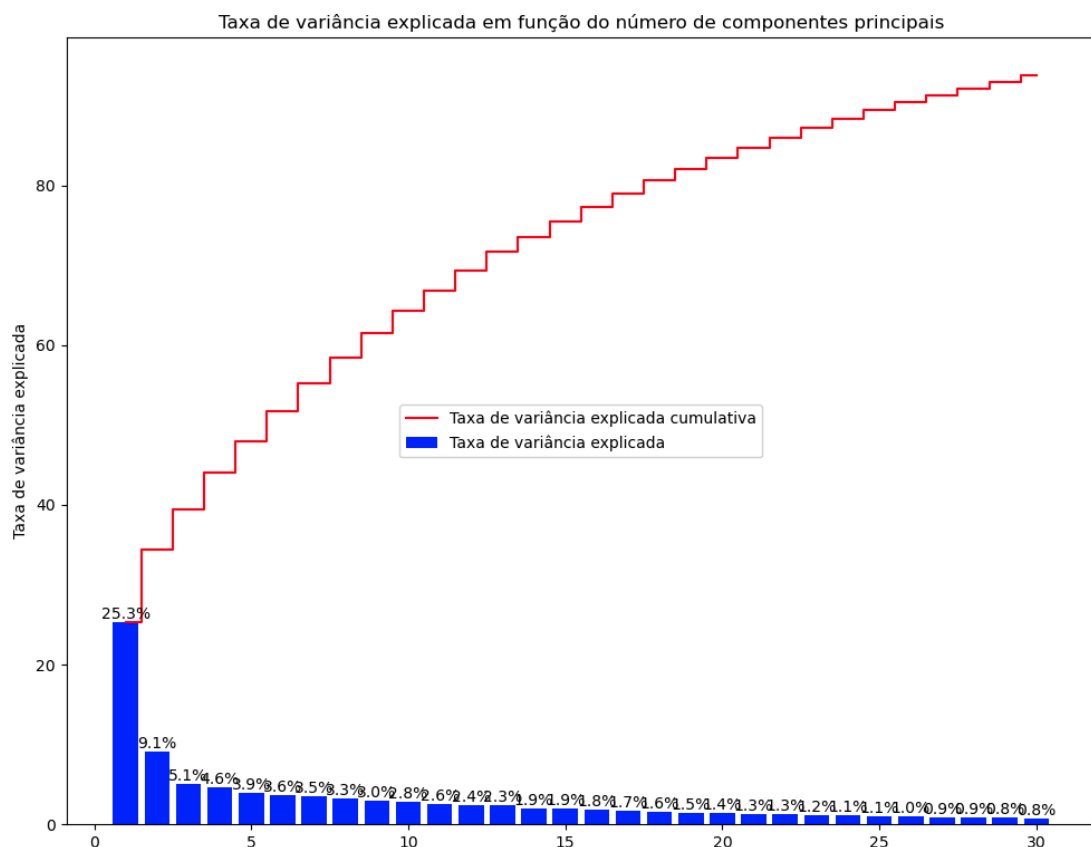
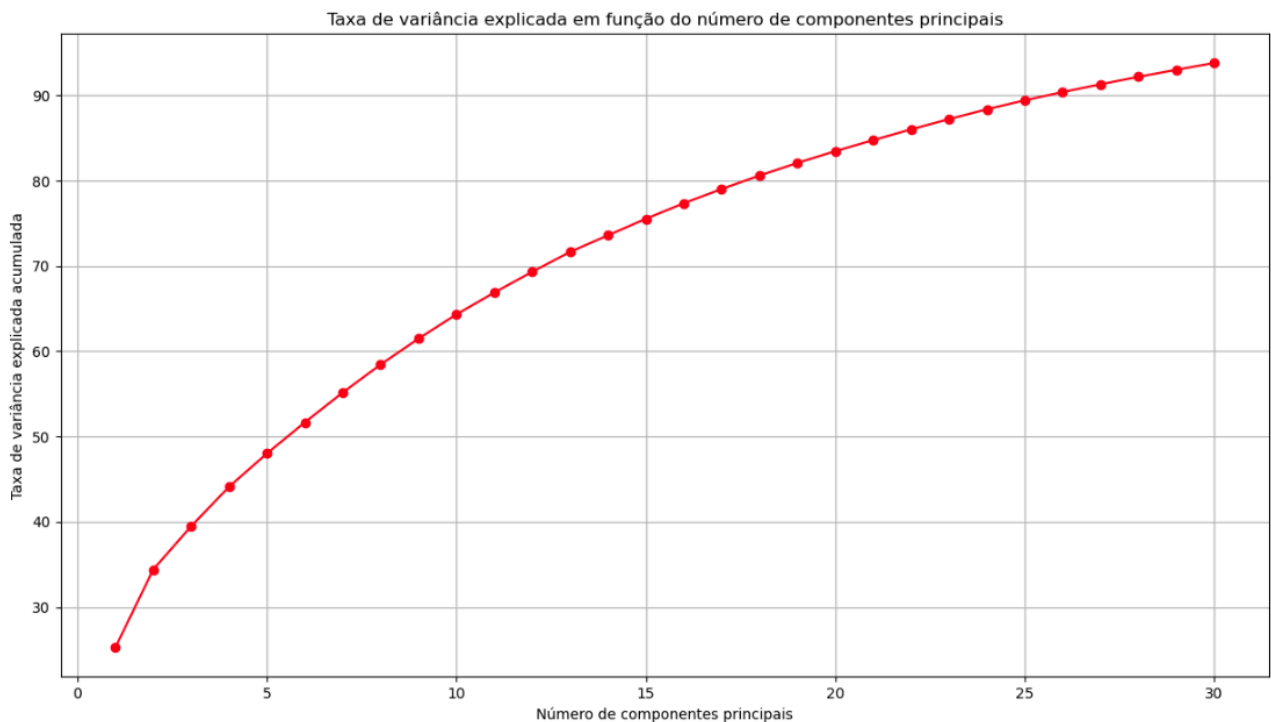
Ao reduzir a dimensionalidade dos nossos dados, nosso objetivo é ter o mínimo de colunas possível com a maior taxa total de variância possível, ou seja, com a maior taxa de variância acumulada por todos os componentes principais (PC) juntos. Conseguir manter uma taxa de variância acima dos 90% após a redução da dimensionalidade é um bom parâmetro.

Portanto, uma das formas de decidirmos um número ideal n de componentes principais para o PCA é justamente testando diferentes valores para n para ver quanto da variância total do dataset eles conseguem representar.

E foi essa técnica que eu apliquei. Para um dataset que já estava com 52 colunas, testei valores entre 1 e 30 para o meu número de componentes principais e fiz somas para chegar à variância total acumulada para cada valor de n .

Colocar esses valores em um gráfico também ajuda a encontrar visualmente o melhor valor de n , normalmente naquele ponto em que a curva não fica mais tão ascendente conforme se aumenta a quantidade de componentes principais.

Optei por plotar dois gráficos distintos: um confrontando a taxa de variância acumulada contra o número de componentes principais, e outro gráfico de barras mostrando a razão de variância aplicada de cada um desses componentes individualmente.



Visualizar a contribuição individual de cada componente principal também é importante pois nos ajuda a ver aqueles que têm uma razão de variância explicada igual ou superior a 1%, que seriam, de acordo com a regra de Kaiser, importantes de serem mantidos no número total de componentes.

5.3. Escolha do número de componentes principais:

Após a análise dos gráficos e levando em conta o padrão de mercado de manter de 80% a 95% da variância total do dataset, bem como a regra de Kaiser de incluir todos os componentes com razão de variância explicada igual ou acima de 1%, optei por definir um n de 25 componentes.

Os 25 componentes juntos conseguem representar quase 90% da variância total da base e somente os 26 primeiros componentes mostraram uma contribuição individual de 1% ou mais para a taxa de variância acumulada. Para arredondar, escolhi o número final de 25, que satisfaz as recomendações de mercado.

6. MODELAGEM

6.1. Seleção dos algoritmos:

Antes de falarmos sobre a escolha dos algoritmos em si, vamos relembrar algumas das características do nosso problema:

- Temos um problema de classificação, isto é, queremos classificar um dado como pertencente a uma determinada classe;
- Nossa classificação é binária, isto é, ou ele é *True* para o diagnóstico de SOP, ou é *False*;
- A base de dados disponível para treino dos algoritmos é relativamente pequena (apenas 684 amostras, sendo que algumas precisaram ser excluídas);
- As colunas das variáveis preditoras mesclam dados categóricos e contínuos;
- Problemas de diagnóstico de condições médicas costumam ter relações não lineares entre as variáveis, isto é, não necessariamente quanto maior o valor de uma variável, maior a probabilidade de ter uma doença; muitas vezes, aquela

variável só tem relevância para o diagnóstico se combinada com a presença de outras variáveis.

Dados esses pontos, é claro que estamos procurando por um algoritmo de **classificação**. Exemplos deles são a Regressão Logística, o Naive Bayes, o K-nearest-neighbors (KNN), as árvores de decisão e etc.

Uma das nossas maiores dificuldades com o dataset escolhido é justamente o baixo número de amostras, que pode levar a problemas de *overfitting* para alguns algoritmos de classificação, como por exemplo a Regressão Logística. Para enfrentar esse problema, a escolha mais certa pareceu ser um modelo de ensemble baseado em árvores de decisão.

Algoritmos como o ***Random Forest Classifier*** são robustos e funcionam muito bem mesmo quando há problemas clássicos, como dataset pequeno, multicolinearidade ou alta dimensionalidade dos dados. O fato de uma floresta aplicar não somente uma, mas centenas de árvores de decisão treinadas com amostras randômicas dos dados reduz muito o risco de *overfitting*, principalmente se dosarmos corretamente a profundidade máxima das árvores. É também provavelmente um dos algoritmos que menos sofre com a multicolinearidade dos dados (comum em problemas de classificação médica e presente no nosso dataset antes da aplicação do PCA), além de exigir pouco pré-processamento das amostras. Por esse motivo, julguei que o ***Random Forest Classifier*** seria um ótimo algoritmo para usar nesse problema.

Outro algoritmo que me pareceu uma boa escolha a princípio foi o da **Regressão Logística**, justamente por ser muito usado como padrão estatístico na medicina e não sofrer tanto risco de *overfitting* em datasets pequenos. Outros pontos a favor desse algoritmo são a sua facilidade de ser explicado, indicando variável a variável quais delas têm mais influência no diagnóstico, e seu resultado com uma probabilidade contínua, que não somente indica se uma amostra pode ou não ser classificada como de um paciente doente, mas ainda traz o percentual exato de chances de aquele paciente ter a condição médica que está sendo avaliada. Infelizmente, no entanto, me deparei com um problema: esse algoritmo parece sofrer bastante interferência quando há multicolinearidade entre as variáveis preditoras, o que é comum em casos médicos e acontece na nossa base; uma forma de resolver o problema da multicolinearidade seria aplicando o PCA, mas isso destrói justamente uma das maiores vantagens da Regressão Logísticas, que é mostrar a influência individual de cada variável no diagnóstico final. Outro problema relaciona-se

com datasets desbalanceados, como é o caso do nosso: quando uma das classes da variável-alvo tem uma incidência muito menor, principalmente em datasets com um número já reduzido de amostras, isso pode levar ao *overfitting* e produzir coeficientes inflacionados. Uma regra clássica da Regressão Logística é ter um **EPV** (*Events Per Variable*) de pelo menos 10 a 20 para cada classe da variável-alvo e, no nosso dataset, a classe minoritária (mulheres sem o diagnóstico de SOP) tem um EPV de inferior a 5, o que compromete o desempenho do algoritmo de Regressão Logística, a não ser que seus hiperparâmetros sejam muito bem regulado. Como ainda estamos aprendendo a configurar hiperparâmetros e a Regressão Logística nem chegou a ser apresentada em aula, eu optei por testar esse algoritmo já imaginando que ele poderia não ter um bom resultado.

Passamos então para o **Support Vector Machine (SVM)**, que funciona muito bem em espaços multidimensionais maiores. Como a base apresentou um bom número de variáveis preditoras, pensei em aplicá-lo sem a remoção prévia das colunas e nem a aplicação do PCA, de modo a conservar uma dimensionalidade mais alta dos dados. Usando um kernel RBF ou polinomial, sua sensibilidade à multicolinearidade dos dados também diminui e ele poderia ter bons resultados.

Por fim, o **K-nearest-neighbors (KNN)** foi escolhido para participar do teste mais para efeito de comparação com os demais algoritmos e para testarmos o efeito que a aplicação do PCA teria no seu desempenho final, pois, devido ao grande número de variáveis categóricas (principalmente binárias), ele provavelmente teria a pior performance entre todos.

6.2. Seleção do pipeline por algoritmo:

Antes de treinar cada algoritmo escolhido, foi preciso primeiro submeter os dados de treino e teste - em conjuntos já separados - ao pipeline correto desejado para cada algoritmo. Para os nossos quatro algoritmos, eu somente quis criar um pipeline diferente para o algoritmo KNN, que precisaria ter os dados submetidos à técnica de PCA para a redução da dimensionalidade dos dados antes do treino.

Para a criação do pipeline, eu primeiro separei as colunas de acordo com o tipo de pré-processamento que elas deveriam receber:

- **colunas categóricas binárias:** como vimos, estas colunas foram codificadas usando o codificador *Ordinal Encoder*;

- **colunas categóricas multiclases:** neste caso, seria necessário usar o codificador *One Hot Encoder*;
- **colunas contínuas:** estas colunas não precisam ser codificadas, mas sim escalonadas com normalização usando o *Min Max Scaler*.

Estes “subpipelines” para cada conjunto de colunas foram agrupados como a etapa de *preprocessor*, comum aos dois pipelines finais.

Para a remoção das colunas com muito baixa correlação com a nossa variável-alvo, foi criado um *transformer* customizado, usando o *Function Transformer* do scikit-learn. Esta etapa foi considerada a etapa **inicial** de cada pipeline final.

Por fim, os pipelines finais foram montados unindo as etapas de remoção de colunas e de pré-processamento dos dados. Um dos pipelines adicionou ainda uma terceira etapa de aplicação do PCA para a redução da dimensionalidade dos dados. Este pipeline, no entanto, foi aplicado somente para o algoritmo KNN.

6.3. Seleção dos hiperparâmetros:

É notável que os algoritmos da biblioteca Scikit-learn têm, de modo geral, uma série de hiperparâmetros que podem ser configurados pelos cientistas de dados manualmente. No entanto, como ainda estamos aprendendo a fazer uso desses algoritmos e para os propósitos deste estudo, entendi que o melhor seria manter essa configuração o mais simples possível, alterando somente alguns hiperparâmetros mais comuns e possivelmente significativos para a diferença no desempenho dos algoritmos.

Para a seleção de alguns hiperparâmetros ajustados, foram inicialmente feitos testes com diferentes valores possíveis, medindo a taxa de erro para cada um deles para decidir quais utilizar para o treinamento. Vamos ver a seguir quais hiperparâmetros foram configurados para cada algoritmo e por quê:

- **Regressão Logística:**
 - *max_iter=1000*: altera o número máximo de iterações para o otimizador convergir. O padrão, que é 100, pode acabar dando avisos de não convergência, o que de fato aconteceu durante o treino do modelo e me fez configurar esse hiperparâmetro;

O hiperparâmetro **penalty** teve seus valores testados (l1, l2 e *elasticnet*) para entender qual deles funcionaria melhor para um dataset com um pouco de multicolinearidade. Quando o valor *elasticnet* foi aplicado, precisamos testar também diferentes valores para o hiperparâmetro **l1_ratio**. Por fim, optei por seguir com o padrão do algoritmo (*penalty='l2'*), pois mostrou ser mais eficiente.

- **K-nearest-neighbors (KNN):**

Para este algoritmo, o único hiperparâmetro que eu configurei foi o **n_neighbors**, que é justamente o número *n* de vizinhos mais próximos que serão selecionados pelo algoritmo para definir a qual classe o novo dado pertence. Testei somente os valores ímpares de 1 a 9, porque valores muito grandes de *n* podem suavizar demais as fronteiras de decisão levando a um *underfitting*. A escolha por testar somente valores ímpares se deu porque nossa variável-alvo é binária, e valores pares de *n* podem acabar resultando em empate na hora da votação que o algoritmo faz para decidir a qual classe o dado pertence.

- **Support Vector Machine (SVM):**

O hiperparâmetro **kernel** foi o que teve seus valores testados ("*linear*", "*poly*", "*rbf*" e "*sigmoid*"). Como os dados do nosso dataset são **não-lineares**, com maior complexidade, era esperado que outras funções de kernel que não a linear contribuíssem para um melhor desempenho do algoritmo. De fato, a função polinomial ("*poly*") foi a escolhida por ter apresentado a menor taxa de erro entre todas.

- **Random Forest Classifier:**

- *class_weight='balanced_subsample'*: a diferença aqui é que usamos o valor de "*balanced_subsample*" para que os pesos fossem recalibrados para cada subgrupo de cada árvore da floresta;
- *n_estimators=100*: o número de árvores para compor nossa floresta. Como nosso número de amostras não é muito grande, optei por 100 árvores, que já seriam suficientes para um bom resultado sem um custo computacional alto;
- *max_depth=200*: a profundidade máxima de cada árvore. É muito importante definir um valor para este hiperparâmetro para evitar que as árvores se ramifiquem demais levando a um possível *overfitting*;

- `random_state=42`: neste tipo de algoritmo, há alguns passos que introduzem aleatoriedade (isto é, são randômicos, e não determinísticos), como o corte aleatório na seleção de variáveis, ou a seleção aleatória de colunas para serem analisadas. É importante definir um *integer* para o `random_state` para garantir as mesmas seleções a cada vez que o algoritmo rodar.

Desta vez, foi o hiperparâmetro ***criterion*** que teve os seus valores testados (somente “*gini*” e “*entropy*”). Este hiperparâmetro define o critério ou métrica de impureza escolhida para definir como os nós da árvore serão criados. No caso, testamos o Índice de Gini e a entropia, e o primeiro foi o que desempenhou melhor.

7. AVALIAÇÃO DOS MODELOS

7.1. Métricas utilizadas:

Para algoritmos classificadores, as principais métricas de avaliação são: **acurácia** (a métrica padrão para esse tipo de algoritmo), **precisão (*precision*)**, **revocação (*recall*)** e ***F-1 Score***.

De maneira geral, é sempre bom dar uma olhada em como cada modelo desempenhou em cada uma dessas métricas, mas é comum também definir uma métrica como a mais importante.

Embora a acurácia seja a métrica mais simples e mais comum entre os algoritmos classificadores, nosso problema é bastante particular. Em primeiro lugar, porque o nosso dataset, como já falamos várias vezes, tem uma **frequência bastante desbalanceada** entre as classes da nossa variável-alvo, o que torna o ***F-1 Score*** mais adequado do que a acurácia para avaliar nossos modelos, uma vez que, em vez de calcular a média aritmética simples entre *precision* e *recall*, ele calcula a **média harmônica**, penalizando muito mais o resultado final quando *precision* ou *recall* forem muito próximas de 0.

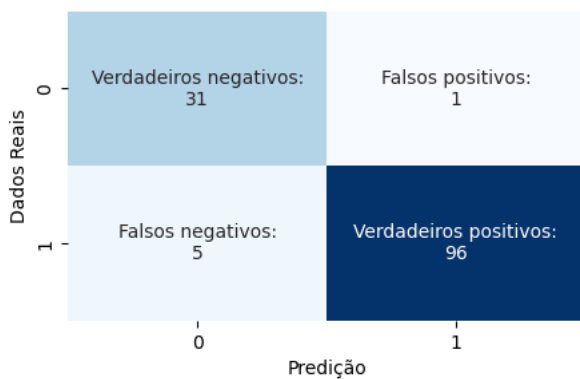
Além disso, não podemos nos esquecer de que estamos tratando de um problema de **diagnóstico médico**. Nestes casos, problemas de **falsos negativos** têm um peso muito maior porque podem deixar passar um diagnóstico de uma doença que pode acabar ficando sem tratamento por um erro de classificação do modelo. Pensando nisso, a métrica ***recall*** ganha um peso muito importante na avaliação dos modelos criados.

Para visualizar todas as quatro métricas citadas, optamos por fazer tanto a matriz de confusão - que nos mostra detalhadamente o desempenho dos modelos para verdadeiros positivos, falsos positivos, verdadeiros negativos e falsos negativos - quanto o *classification report*, que traz o resumo de todas as métricas em uma tabela super eficiente.

7.2. Resultados comparativos:

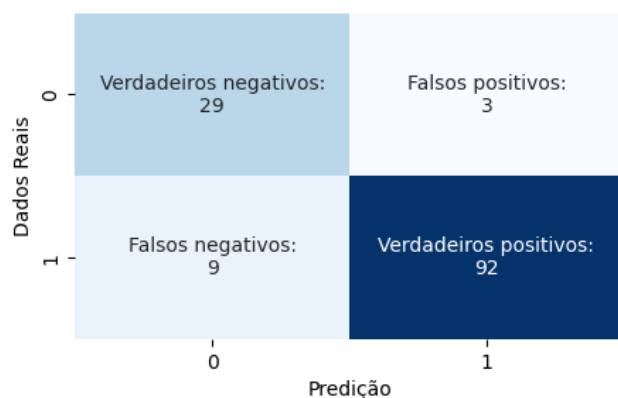
Abaixo, acompanhe o resultado de cada um dos modelos testados por sua matriz de confusão e *classification report*:

- Regressão Logística:



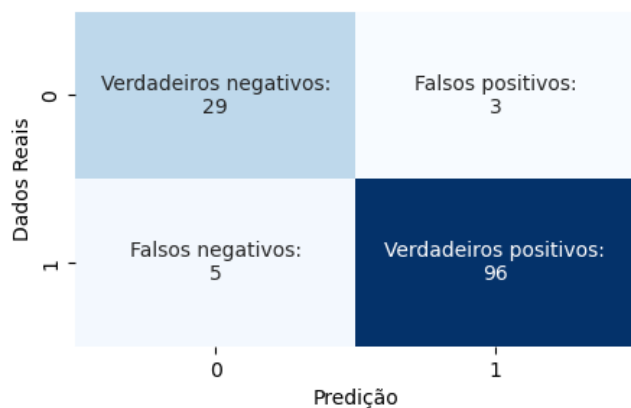
	precision	recall	f1-score	support
Sem SOP	0.86	0.97	0.91	32
Com SOP	0.99	0.95	0.97	101
accuracy			0.95	133
macro avg	0.93	0.96	0.94	133
weighted avg	0.96	0.95	0.96	133

- *K-nearest-neighbors* (KNN):



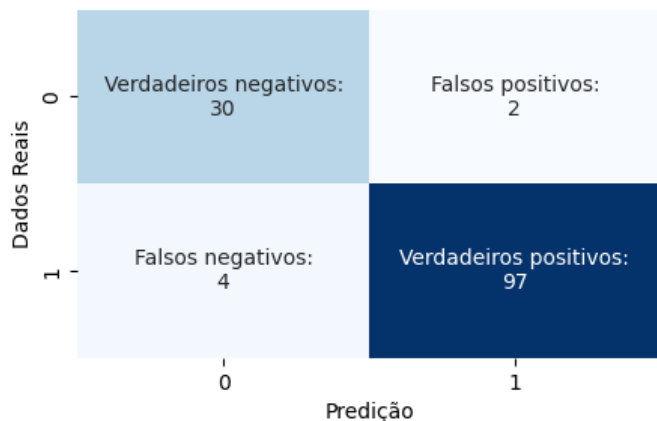
	precision	recall	f1-score	support
Sem SOP	0.76	0.91	0.83	32
Com SOP	0.97	0.91	0.94	101
accuracy			0.91	133
macro avg	0.87	0.91	0.88	133
weighted avg	0.92	0.91	0.91	133

- *Support Vector Machine (SVM):*



SVC	precision	recall	f1-score	support
Sem SOP	0.85	0.91	0.88	32
Com SOP	0.97	0.95	0.96	101
accuracy			0.94	133
macro avg	0.91	0.93	0.92	133
weighted avg	0.94	0.94	0.94	133

- *Random Forest Classifier:*



	precision	recall	f1-score	support
Sem SOP	0.88	0.94	0.91	32
Com SOP	0.98	0.96	0.97	101
accuracy			0.95	133
macro avg	0.93	0.95	0.94	133
weighted avg	0.96	0.95	0.96	133

7.3. Discussão dos resultados:

Como já era esperado desde o momento da seleção dos algoritmos, o algoritmo que teve o melhor desempenho foi de fato o **Random Forest Classifier**. Ele teve o melhor desempenho em todas as métricas, com um excelente valor de *recall* para a classificação da doença (96%), além de um *F1-Score* também bastante elevado (97%). Para nós, o mais interessante é que ele teve somente 4 falsos negativos entre as 133 amostras do conjunto de teste, o que é desejado em modelos para a classificação de condições médicas.

Olhando para os demais algoritmos, a Regressão Logística surpreendeu positivamente, com desempenho muito próximo ao do *Random Forest Classifier*, com acurácia e *recall* de 95%, e um *F1-Score* de 97% para a classe com o diagnóstico, mesmo com a limitação teórica do baixo EPV. Isso sugere que, apesar da

multicolinearidade presente no dataset, a relação entre as variáveis mais relevantes e o diagnóstico de SOP é forte o suficiente para que até um modelo linear simples capture bem o padrão.

O DVM com o kernel polinomial também teve bom desempenho, com *recall* de 95%, validando a hipótese de que a natureza não linear dos dados se beneficiaria de um kernel diferente do padrão (linear) e reforçando a decisão de mantê-lo com uma dimensionalidade mais alta, sem a aplicação do PCA.

Já o KNN, conforme o esperado, teve o pior desempenho entre os quatro algoritmos testados, mas ainda assim um desempenho excelente, com acurácia e *recall* de 91% e apenas 9 falsos negativos.

Comparando os falsos negativos de todos os algoritmos - a métrica que mais nos importa neste problema por se tratar de diagnóstico médico - o *Random Forest Classifier* teve o melhor resultado (apenas 4), seguido de perto pela Regressão Logística e pelo SVM (5 falsos negativos cada), enquanto o KNN teve o pior desempenho, com 9 falsos negativos. Isso reforça o *Random Forest Classifier* como a escolha mais segura para este tipo de aplicação, mas também mostra que Regressão Logística e SVM poderiam ser boas alternativas, com a vantagem adicional de serem mais fáceis de explicar e com menor custo computacional.

8. CONCLUSÃO

8.1. Modelo recomendado e justificativa:

Entre os quatro algoritmos testados, o *Random Forest Classifier* é o modelo recomendado para este problema. Ele apresentou o melhor desempenho em todas as métricas avaliadas (acurácia de 95%, *F1-Score* de 97% e *recall* de 96% para a classe com o diagnóstico de SOP), além do menor número de falsos negativos (4 em 133 amostras de teste) — o critério mais crítico neste contexto, já que um falso negativo representa uma paciente com SOP que o modelo classificaria erroneamente como saudável, atrasando um diagnóstico e consequente tratamento.

Além do desempenho, o *Random Forest* tem vantagens práticas relevantes para este dataset: lida bem com a multicolinearidade natural entre variáveis clínicas, é pouco sensível à alta dimensionalidade gerada pela codificação *One Hot Encoding*, e é mais

resistente a *overfitting* em bases pequenas do que algoritmos mais simples, como a Regressão Logística. A interpretabilidade obtida com o *SHAP* também é um ponto a favor, pois permite validar clinicamente as variáveis mais influentes na decisão do modelo, sem abrir mão do desempenho.

8.2. Limitações do projeto:

- **Tamanho do dataset:** com apenas 684 amostras (reduzidas ainda mais após a remoção de outliers), a base é pequena para tarefas de machine learning, especialmente para um problema de saúde com potencial variabilidade populacional. O conjunto de teste (133 amostras) é pequeno o suficiente para que poucas amostras mal classificadas alterem sensivelmente as métricas;
- **Desbalanceamento da variável-alvo:** mais de 75% das amostras correspondem a pacientes diagnosticadas com SOP, o que é o oposto do que se observa na população geral (10% a 13% de prevalência). Isso levanta a hipótese de viés na coleta dos dados (por exemplo, um dos datasets de origem pode ter sido coletado em contexto clínico, favorecendo mulheres já com suspeita da condição) e limita a capacidade de generalização do modelo para uma população real, onde a proporção é invertida;
- **Dados ausentes:** 9 das 32 variáveis preditoras, em sua maioria exames laboratoriais, tinham alta taxa de dados ausentes (entre 50 e 252 amostras sem valor), concentrados majoritariamente entre as mulheres sem diagnóstico de SOP. Optar por manter esses dados como uma categoria ("*Missing*") evitou introduzir viés por imputação simplista (como a moda), mas não deixa de ser uma limitação: o modelo pode estar aprendendo, em parte, um padrão de "ausência de solicitação médica de exames" em vez de um padrão puramente clínico da doença.

8.3. Possíveis próximos passos:

- Aplicar validação cruzada com *GridSearchCV* (ideia já cogitada durante o projeto, mas não implementada), permitindo testar múltiplas combinações de hiperparâmetros de forma mais rigorosa e aproveitar melhor a base de dados pequena;
- Buscar ampliar e/ou balancear a base de dados, seja coletando mais amostras de pacientes sem SOP, seja aplicando técnicas de balanceamento como oversampling (SMOTE) ou undersampling controlado;

- Investigar técnicas mais sofisticadas de tratamento de dados ausentes (como imputação múltipla ou modelos preditivos para os valores faltantes), especialmente se uma base maior estiver disponível;
- Validar o modelo escolhido em uma base de dados externa (outra amostra populacional), para avaliar sua real capacidade de generalização fora do contexto de coleta original;